TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: HAMILTON ROGER MOSQUERA GONZALEZ

PARALELO: A

SEMESTRE: QUINT

ACTIVIDAD: #1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad

Actividad #1: ***Estudio de Caso - Sistema de Control de Rescate y Adopción Animal***

1. Introducción

La gestión de animales rescatados representa un proceso que requiere organización, seguimiento y coordinación entre ciudadanos, voluntarios, veterinarios y administradores de refugios. Cuando la información se maneja de manera manual o mediante registros dispersos, pueden presentarse dificultades para identificar a los animales, asignar responsables, conocer el estado de atención y conservar el historial de cada caso.

Ante esta problemática, se propone el diseño lógico de un Sistema de Control de Rescate y Adopción Animal, una solución informática orientada a centralizar la información de los animales rescatados y organizar las actividades relacionadas con su atención, tratamiento y posible adopción.

El sistema permitirá registrar casos de rescate, clasificar a los animales según su especie, asignar voluntarios o veterinarios y controlar los cambios de estado del proceso. Para atender perros, gatos y fauna silvestre, se plantea una estructura que facilite la incorporación de información específica sin duplicar innecesariamente la lógica general del sistema.

El diseño aplicará los patrones de diseño Singleton, Observer, Factory Method y Model-View-Controller (MVC), seleccionados por su utilidad para administrar los casos, notificar eventos, crear expedientes especializados y separar las responsabilidades de la aplicación. Asimismo, se propondrá el uso de Gitflow para organizar el desarrollo y mantener la trazabilidad de los cambios. Para la representación gráfica del diseño se utilizará PlantText.com, una herramienta en línea que permite elaborar diagramas UML mediante código PlantUML, facilitando la visualización de la estructura, las relaciones y el comportamiento de los componentes del sistema. (PlantText, 2026)

Este trabajo se concentra en el análisis y diseño de la solución, estableciendo una base técnica coherente para una futura implementación.

2. Planteamiento del problema

En un refugio animal pueden recibirse reportes de animales abandonados, heridos o en situación de riesgo. Cada reporte requiere registrar información, identificar la especie, conocer la ubicación del rescate, determinar las necesidades de atención y asignar a una persona responsable.

Si no existe un sistema centralizado, la información puede quedar incompleta o desactualizada. También puede resultar difícil conocer quién está atendiendo cada caso, consultar los cambios de estado y comunicar nuevos rescates a los voluntarios disponibles.

La situación se vuelve más compleja cuando se gestionan especies diferentes, ya que un perro, un gato y un animal silvestre pueden necesitar datos particulares en sus respectivos expedientes.

- Por ello, se plantea diseñar una solución que permita:
- Centralizar los reportes y registros de rescate.
- Diferenciar los animales por especie.
- Asignar responsables para las actividades de atención.
- Registrar y consultar el historial de los casos.
- Generar notificaciones cuando se registra un nuevo rescate.
- Facilitar el mantenimiento y la ampliación del software.
- Controlar los cambios realizados durante el desarrollo mediante Gitflow.

3. Objetivos

3.1. Objetivo general

Diseñar la arquitectura lógica de un Sistema de Control de Rescate y Adopción Animal mediante el análisis de requisitos, la elaboración de diagramas UML, la aplicación de patrones de diseño y la planificación del flujo de trabajo Gitflow, con el propósito de establecer una solución organizada, escalable y mantenible.

3.2. Objetivos específicos

1. Identificar los actores, las necesidades y los requerimientos funcionales y no funcionales del sistema.
2. Representar los procesos y las relaciones del sistema mediante diagramas UML de casos de uso, clases y secuencia.
3. Aplicar los patrones Singleton, Observer, Factory Method y MVC, justificando su utilidad dentro de la solución propuesta.
4. Incorporar principios SOLID para promover una separación adecuada de responsabilidades y facilitar el mantenimiento del software.
5. Diseñar una arquitectura lógica que relacione la interfaz, la lógica de negocio, la persistencia de datos y el proceso de control de versiones.

4. Alcance del sistema

El sistema propuesto permitirá gestionar la información de los casos de rescate desde su registro hasta el seguimiento de la atención y el proceso de adopción cuando corresponda.

Módulo	Función principal
Registro de rescates	Registrar reportes, ubicación, fecha y descripción del caso.
Gestión de animales	Registrar identificación, nombre si se conoce, especie y condición.
Expedientes por especie	Guardar datos particulares de perros, gatos y fauna silvestre.
Asignaciones	Relacionar casos con voluntarios o veterinarios responsables.
Seguimiento	Registrar los cambios de estado y conservar el historial.
Notificaciones	Informar a los observadores suscritos sobre nuevos casos.
Administración	Gestionar usuarios, permisos y consultas generales.

Persistencia	Guardar y recuperar los datos de manera organizada.

4.1. Fuera del alcance inicial

Para mantener el diseño controlado, la primera versión no incluirá necesariamente pagos, geolocalización en tiempo real, una aplicación móvil independiente, integración con sistemas veterinarios externos ni un algoritmo automático de selección de adoptantes. Estas funciones podrían evaluarse en futuras versiones.

4.2. Conceptos fundamentales del dominio

Es importante diferenciar tres elementos:

- Animal: representa al ser vivo y conserva su información de identificación.
- Caso de rescate: representa el proceso de atención asociado a un reporte.
- Expediente del animal: contiene información general y específica de su especie.

Esta separación permite que un animal tenga varios casos a lo largo del tiempo sin duplicar toda su información.

5. Análisis del sistema

5.1. Actores principales

	**Ciudadano / Reportante** Persona que informa sobre un animal que necesita ayuda. En la propuesta puede registrar un reporte y aportar la ubicación, la descripción y la información disponible del animal.
	**Voluntario / Veterinario** Personal que consulta casos, recibe asignaciones, atiende animales y registra observaciones. El veterinario puede documentar la atención clínica, mientras que el voluntario puede apoyar en rescate, traslado y cuidado.

	**Administrador del refugio** Responsable de supervisar los registros, administrar usuarios y permisos, asignar personal, revisar los casos y consultar su evolución.

5.2. Requerimientos funcionales

Los requerimientos funcionales describen las operaciones que debe permitir el sistema. Los siguientes son una propuesta de diseño para cumplir el caso planteado.

Código	Requerimiento funcional	Actor principal
RF01	Registrar un reporte de rescate con fecha, ubicación y descripción.	Ciudadano
RF02	Generar un identificador único para cada caso.	Sistema
RF03	Registrar los datos de identificación del animal.	Administrador / Personal
RF04	Clasificar el animal como perro, gato o fauna silvestre.	Administrador / Personal
RF05	Crear un expediente según la especie del animal.	Sistema
RF06	Consultar los casos registrados.	Personal / Administrador
RF07	Asignar un voluntario o veterinario responsable a un caso.	Administrador
RF08	Consultar las asignaciones de cada responsable.	Voluntario / Veterinario
RF09	Cambiar el estado del caso según la evolución de la atención.	Personal autorizado
RF10	Conservar el historial de cambios de estado.	Sistema
RF11	Notificar la creación de un nuevo caso a los observadores suscritos.	Sistema

RF12	Registrar observaciones sobre la atención del animal.	Voluntario / Veterinario
RF13	Registrar la información relacionada con el proceso de adopción.	Administrador / Personal autorizado
RF14	Administrar usuarios y permisos de acceso.	Administrador
RF15	Consultar el historial y el estado actual de un caso.	Personal autorizado

5.3. Requerimientos no funcionales

Código	Requerimiento	Criterio propuesto
RNF01	Mantenibilidad	Separar la interfaz, la lógica de negocio y la persistencia.
RNF02	Escalabilidad	Permitir incorporar nuevos módulos y especies sin reescribir toda la aplicación.
RNF03	Trazabilidad	Registrar los cambios del código mediante Git y GitHub.
RNF04	Seguridad	Restringir las operaciones de acuerdo con los permisos del usuario.
RNF05	Integridad	Validar los datos antes de guardar un caso o una asignación.
RNF06	Disponibilidad de información	Permitir consultar los casos almacenados por usuarios autorizados.
RNF07	Usabilidad	Presentar formularios y mensajes comprensibles.
RNF08	Consistencia	Mantener relaciones válidas entre animales, casos, asignaciones y expedientes.

RNF09	Extensibilidad	Permitir agregar tipos de expedientes sin alterar las operaciones comunes.
RNF10	Auditoría del proceso	Conservar fechas, responsables y cambios de estado relevantes.

Los criterios anteriores son objetivos de diseño propuestos. Para una implementación real se tendrían que establecer métricas verificables, como tiempos máximos de respuesta, disponibilidad esperada y políticas concretas de seguridad.

5.4. Reglas de negocio

Las reglas de negocio determinan las condiciones que deben respetarse para que la información sea coherente.

RN01. Identificación: cada caso de rescate debe poseer un identificador único.
RN02. Relación con el animal: cada caso debe estar asociado a un animal registrado.
RN03. Clasificación: la especie debe corresponder a una categoría admitida: perro, gato o fauna silvestre.
RN04. Asignación: un caso que requiera atención operativa debe contar con un responsable asignado. El sistema podrá permitir que el caso recién reportado permanezca temporalmente sin asignación, siempre que se encuentre pendiente de atención.
RN05. Estados: los cambios de estado deben validarse y quedar registrados en un historial.
RN06. Notificaciones: una vez guardado correctamente un nuevo caso, se debe generar el evento de notificación a los observadores suscritos.
RN07. Permisos: las operaciones administrativas solo pueden ser ejecutadas por usuarios autorizados.
RN08. Adopción: la transición al estado de adopción debe respetar las validaciones que establezca el refugio.
RN09. Asignaciones: cada asignación debe identificar el caso, el responsable, la fecha y la función que cumplirá.

RN10. Expedientes: el expediente debe contener los datos comunes y permitir información específica según la especie.

La guía menciona estados como «Rescatado», «En Tratamiento» y «Adopción». Los demás estados son una ampliación propuesta para representar el ciclo de atención con mayor claridad.

No se debe permitir pasar directamente de «Reportado» a «Adoptado» sin completar las validaciones que defina el refugio. Asimismo, los estados terminales, como «Adoptado» o «Cerrado», deben impedir modificaciones ordinarias que alteren el historial.

6. Diagrama UML de casos de uso

Este diagrama representa las acciones principales que los actores realizan con el sistema. Los casos de uso están delimitados dentro del sistema y se conectan con los actores correspondientes.

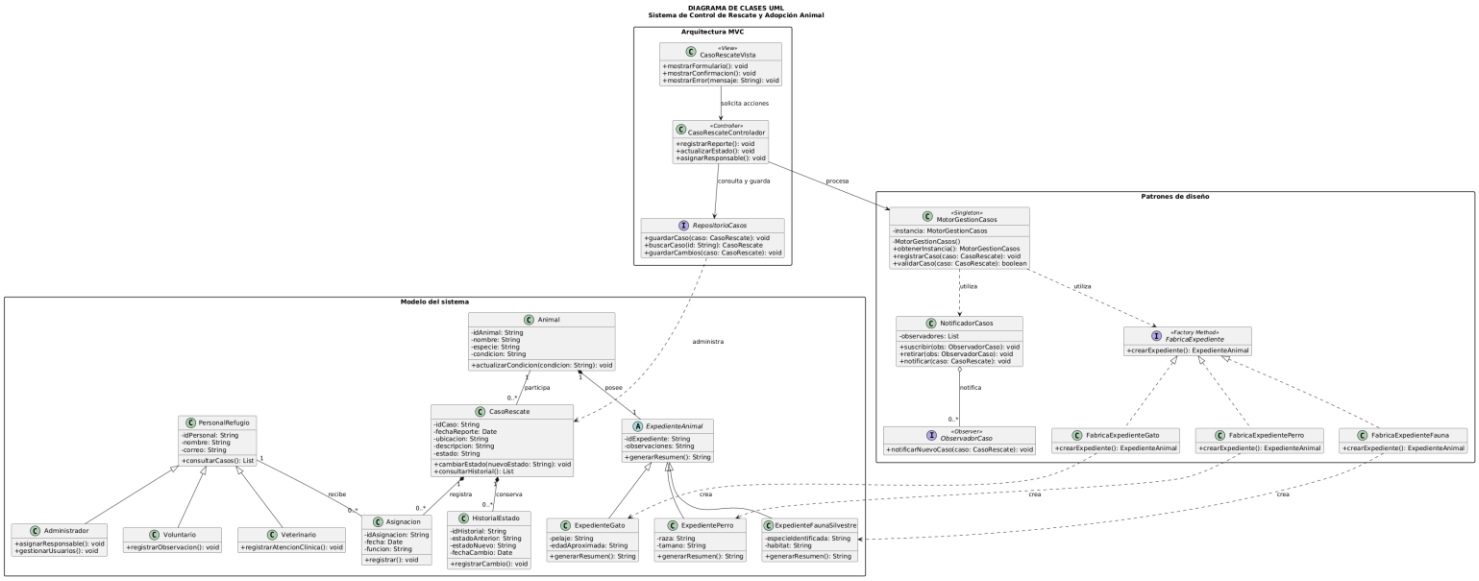


Ilustración 1: El diagrama UML, fue realizado en PlantText, cual es un editor en línea gratuito que funciona como un generador de diagramas a partir de texto plano.

Interpretación del diagrama:

- El ciudadano registra un reporte para informar sobre un animal que necesita ayuda.

- El proceso de registro incluye la clasificación de la especie y la creación del expediente correspondiente.
- El voluntario y el veterinario consultan los casos y registran las observaciones de atención según sus responsabilidades.
- El administrador organiza las asignaciones y gestiona los permisos.

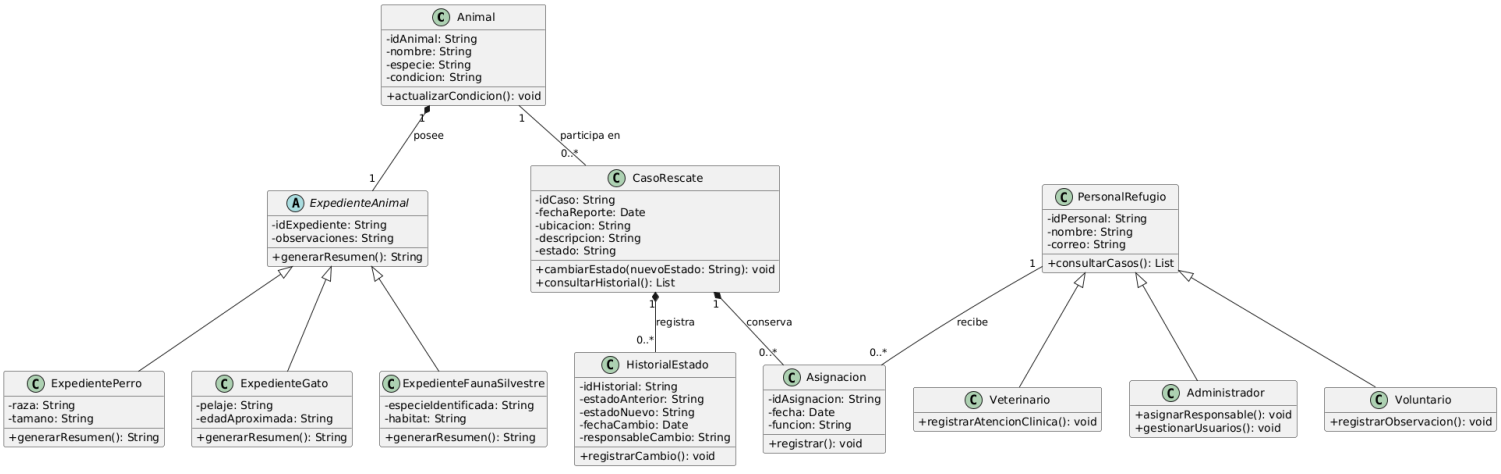
El historial permite consultar la evolución del caso.

La relación include indica que el comportamiento incluido forma parte del proceso principal. La relación extend representa un comportamiento adicional que ocurre bajo determinadas condiciones; en este diseño, la información de adopción puede incorporarse cuando corresponda. El diagrama podrá ajustarse si, al implementar el sistema, se determina que el registro del reporte y la creación del expediente deben separarse en pasos independientes.

7. Diagrama de clases UML

El diagrama de clases representa la estructura estática de la solución: sus entidades, atributos, operaciones y relaciones.

Para evitar confusiones, el modelo distingue Animal, CasoRescate y ExpedienteAnimal. La clase CasoRescate controla el proceso, mientras que el expediente conserva los datos del animal y sus características particulares.



8.1. Explicación de las clases

Clase	Responsabilidad

Animal	Identificar al animal y conservar sus datos generales.
ExpedienteAnimal	Definir los atributos y operaciones comunes de un expediente.
ExpedientePerro	Añadir datos propios de los perros, como raza y tamaño.
ExpedienteGato	Incorporar información específica de los gatos.
ExpedienteFaunaSilvestre	Registrar especie identificada y hábitat cuando se conozcan.
CasoRescate	Gestionar el reporte, estado y evolución del proceso de rescate.
Asignacion	Registrar quién es responsable, desde qué fecha y con qué función.
PersonalRefugio	Representar los datos comunes del personal.
Voluntario	Representar las funciones específicas del voluntario.
Veterinario	Representar las funciones específicas de atención veterinaria.
Administrador	Representar las operaciones administrativas autorizadas.
HistorialEstado	Conservar cada transición de estado, su fecha y responsable.

7.2. Relaciones entre las clases

Herencia: Voluntario, Veterinario y Administrador heredan de PersonalRefugio. Así se reutilizan los datos y comportamientos comunes del personal.
Herencia de expedientes: los expedientes específicos heredan de ExpedienteAnimal, lo que permite tratar los diferentes tipos mediante una abstracción común.
Composición Animal–Expediente: cada animal posee un expediente principal en el modelo propuesto. Si en una implementación se necesitan expedientes históricos, se podrá ampliar esta relación.

Asociación Animal–CasoRescate: un animal puede participar en varios casos a lo largo del tiempo.
Composición CasoRescate–Asignación: las asignaciones pertenecen al caso que describen.
Composición CasoRescate–HistorialEstado: el historial conserva los cambios relacionados con el caso.

7.3. Aplicación de los principios SOLID

Principio	Aplicación en el sistema
S — Responsabilidad única	CasoRescate administra el proceso; Asignacion registra la responsabilidad; HistorialEstado conserva los cambios.
O — Abierto/cerrado	Se pueden añadir nuevas clases de expediente sin reescribir la estructura común, si se programa contra la abstracción ExpedienteAnimal.
L — Sustitución de Liskov	Cada expediente especializado debe cumplir el contrato definido por ExpedienteAnimal.
I — Segregación de interfaces	En una implementación, conviene separar contratos como Notificador, Observador y RepositorioCasos, evitando interfaces demasiado generales.
D — Inversión de dependencias	El motor de gestión debería depender de abstracciones de repositorio y notificación, no directamente de una base de datos o servicio concreto.

el diagrama es una propuesta lógica. Las interfaces de los patrones se incorporarán en el siguiente apartado para hacer explícitas las dependencias necesarias. No es conveniente concentrar todas las operaciones del sistema en una sola clase.

8. Patrones de diseño

La guía solicita al menos tres patrones y recomienda Singleton, Observer, Factory Method y MVC. Se propone utilizar los cuatro porque resuelven problemas distintos dentro de la arquitectura.

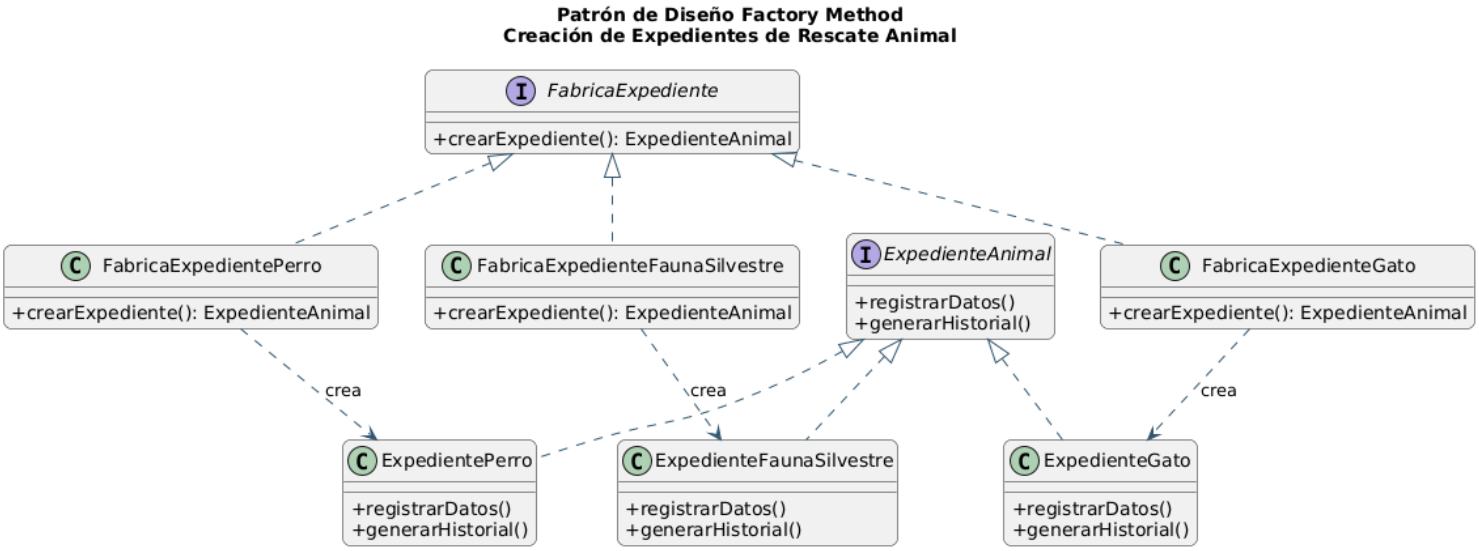
8.1. Singleton

Propósito: controlar la creación de una única instancia de un componente que necesita coordinar una operación central.

Aplicación: se propone MotorGestionCasos como punto de coordinación de las operaciones de gestión de casos. Su instancia única evita que la aplicación cree motores de gestión duplicados de forma accidental.

Problema que resuelve: sin una instancia controlada, diferentes partes del sistema podrían crear motores independientes con configuraciones o dependencias distintas, complicando la coordinación de los procesos.

Diseño ilustrativo en Java:



El constructor privado impide crear objetos mediante new desde otras clases, mientras que obtenerInstancia() entrega la instancia compartida.

Consideración técnica: Singleton no debe utilizarse automáticamente para todos los componentes. El acceso a la base de datos puede gestionarse mediante un conjunto de conexiones o un componente de infraestructura apropiado. En un sistema web, el alcance de la instancia debe evaluarse según el entorno de ejecución; una instancia estática no significa necesariamente que exista una única instancia en todos los servidores.

8.2. Observer

Propósito: notificar a varios objetos interesados cuando sucede un evento, sin obligar al componente que produce el evento a conocer los detalles internos de cada receptor.

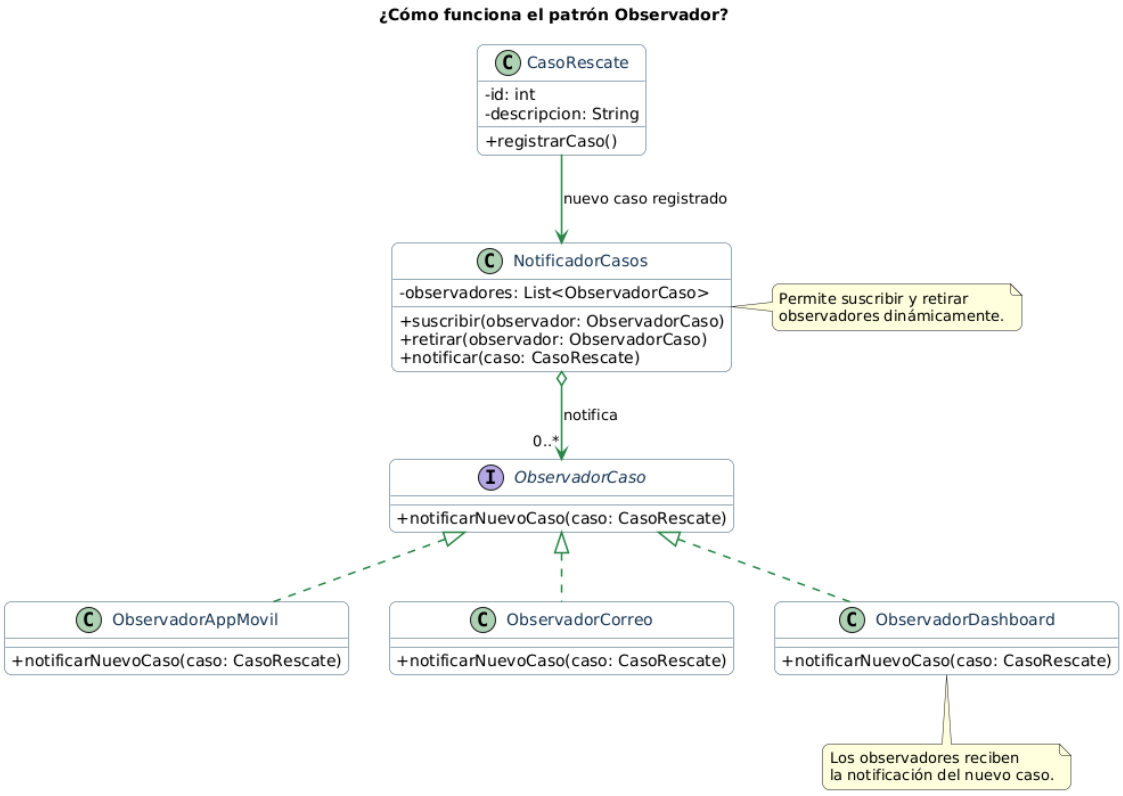
Aplicación: cuando se registra un nuevo caso de rescate, el sistema genera un evento y notifica a los voluntarios u otros destinatarios suscritos.

Se definen dos contratos:

- ObservadorCaso: recibe el aviso.

- NotificadorCasos: permite suscribir, retirar y notificar observadores.

Diseño ilustrativo en Java:



La clase CasoRescate se muestra como referencia del dominio; este fragmento ilustra el mecanismo de suscripción, no una aplicación completa ejecutable por sí sola.

Problema que resuelve: evita que el proceso de registro tenga que contener la lógica de envío a cada voluntario. Si posteriormente se incorpora notificación por correo

electrónico, aplicación móvil o panel interno, se pueden añadir observadores específicos sin cambiar la regla principal de registro.

Regla importante: el evento de notificación debe emitirse después de guardar correctamente el caso. No se deben notificar rescates que hayan fallado al persistirse.

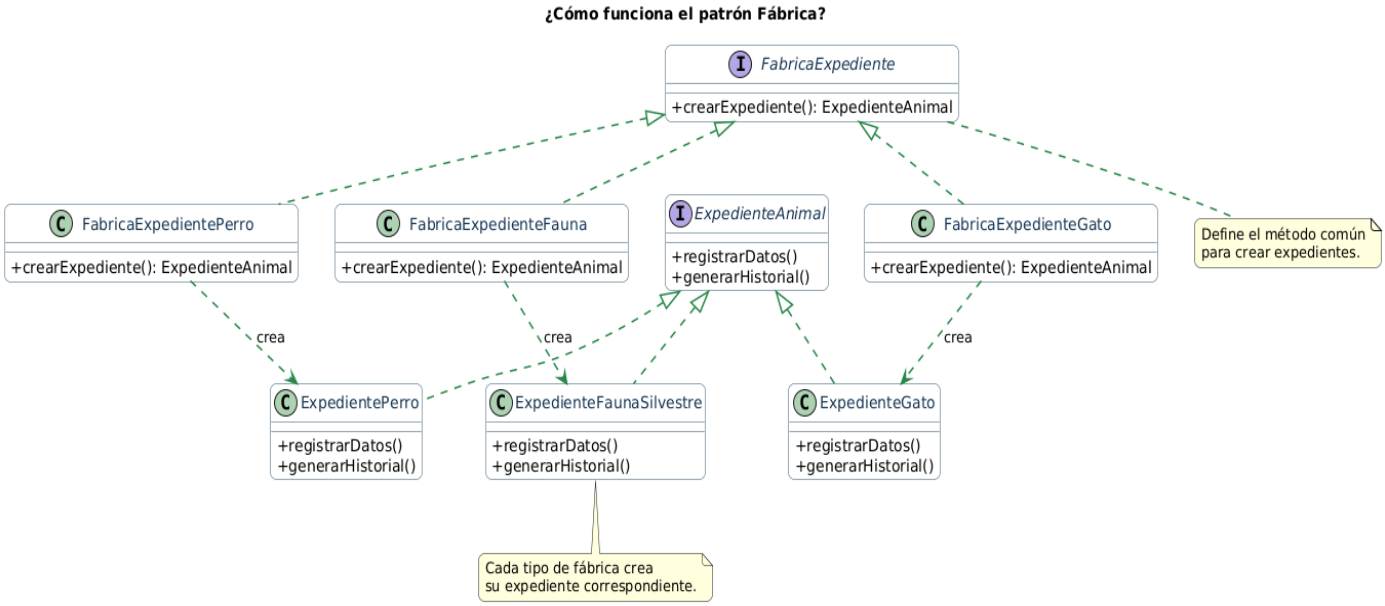
8.3. Factory Method

Propósito: delegar la creación de objetos a métodos especializados, evitando que el código cliente tenga que decidir directamente qué clase concreta instanciar en cada lugar.

Aplicación: crear expedientes según la especie del animal.

El contrato será FabricaExpediente, con una implementación concreta para cada tipo: perro, gato y fauna silvestre.

Diseño ilustrativo en Java:



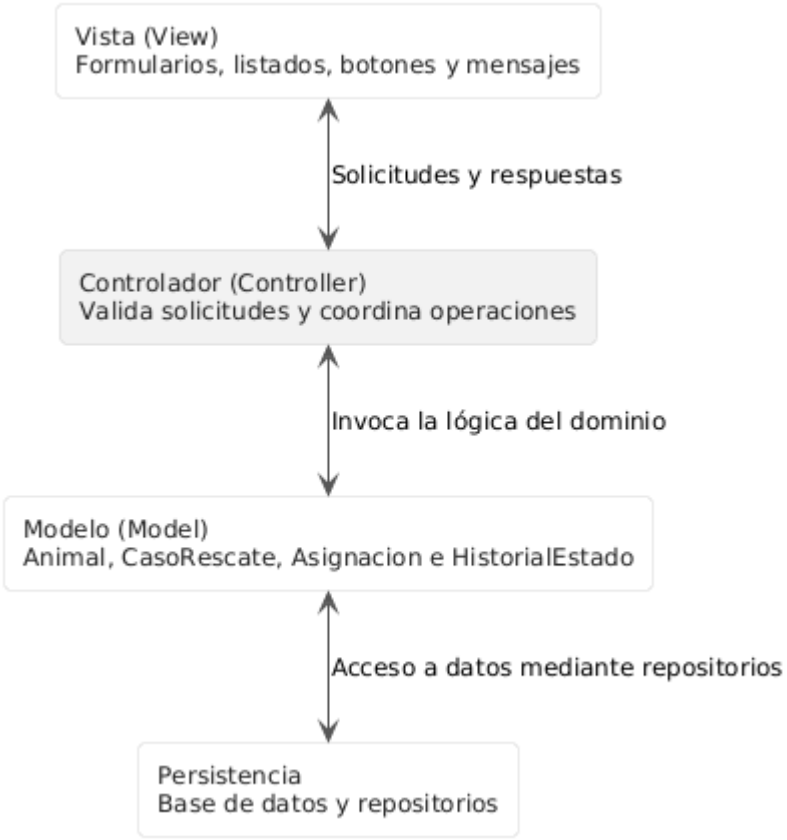
Este ejemplo presupone constructores sin parámetros en las clases concretas. En la implementación real, el método podría recibir datos iniciales o utilizar un objeto de entrada validado.

Problema que resuelve: evita dispersar condicionales como if (especie.equals("perro")) por distintas partes de la aplicación. La creación queda concentrada en fábricas especializadas.

Ventaja de mantenimiento: si se incorpora una nueva categoría de animal, se puede añadir un nuevo expediente y su fábrica, procurando mantener estable el contrato general.

8.4. MVC (Model–View–Controller)

Propósito: separar los datos y reglas del dominio, la presentación y el tratamiento de las solicitudes.



Aplicación propuesta:

CasoRescateVista: muestra el formulario de registro y los detalles de los casos.

CasoRescateControlador: recibe las solicitudes, valida los datos de entrada y coordina el proceso.

CasoRescate: representa las reglas y el estado del caso.

RepositorioCasos: define el contrato para guardar y consultar casos.

RepositorioCasosBD: implementa el acceso a una base de datos concreta.

Problema que resuelve: permite modificar la interfaz sin reescribir las reglas de negocio y cambiar el mecanismo de persistencia sin mezclar instrucciones SQL con los componentes visuales.

MVC es un patrón arquitectónico de presentación, mientras que Singleton, Observer y Factory Method se aplican a problemas de creación y coordinación de objetos. Por eso, los cuatro patrones se complementan en lugar de reemplazarse.

Relación entre los patrones y los requisitos

Patrón	Componente principal	Requisito o necesidad atendida
Singleton	MotorGestionCasos	Coordinación central de operaciones.
Observer	NotificadorCasos	RF11: notificaciones de nuevos rescates.
Factory Method	FabricaExpediente	RF04–RF05: clasificación y creación de expedientes.
MVC	Vista, controlador y modelo	Separación de presentación, lógica y acceso a datos.

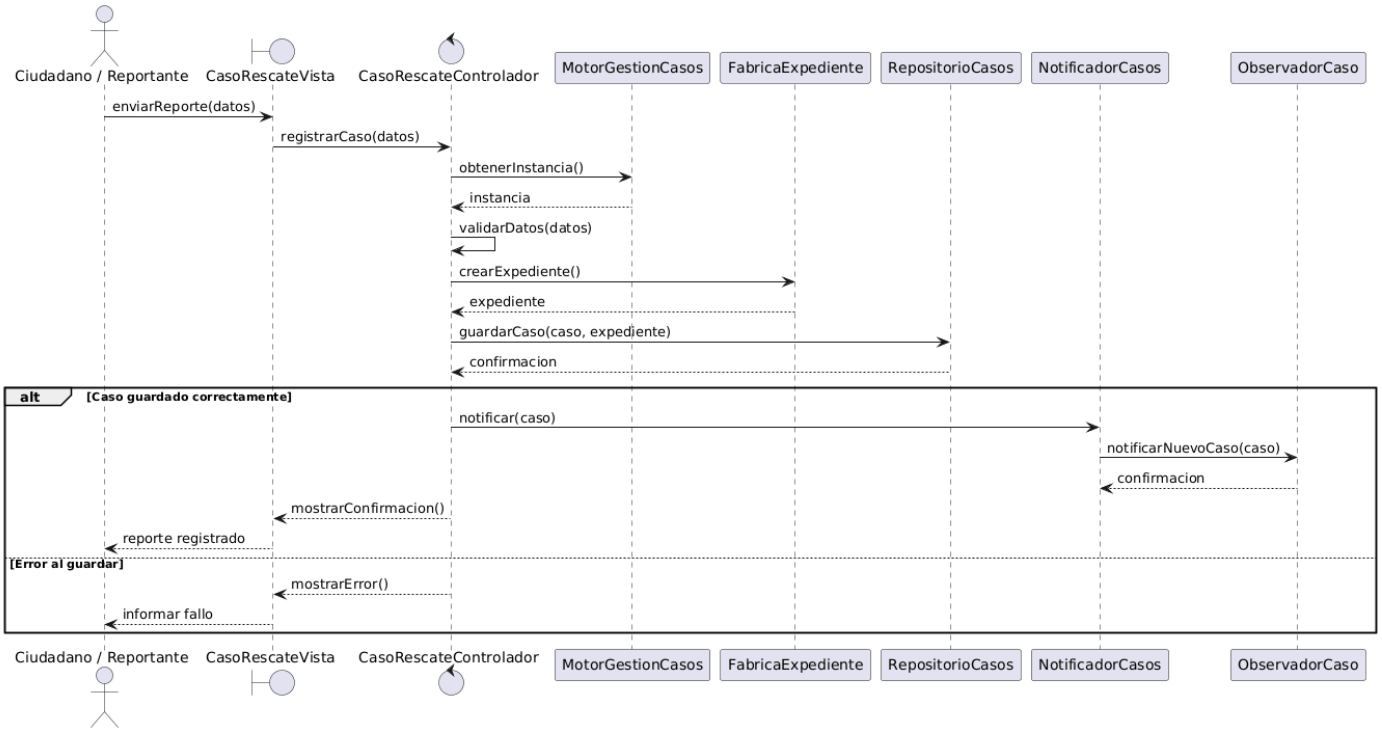
Los patrones describen una forma de organizar el código; no sustituyen las reglas de validación, la seguridad, las pruebas ni el diseño de persistencia.

9. Diagramas de secuencia UML

Los diagramas de secuencia describen cómo interactúan los componentes en un escenario concreto. Para que sean consistentes con los patrones definidos, se utilizarán los mismos nombres de clases y operaciones en los diagramas y en el diseño.

9.1. Secuencia: registrar un nuevo caso de rescate

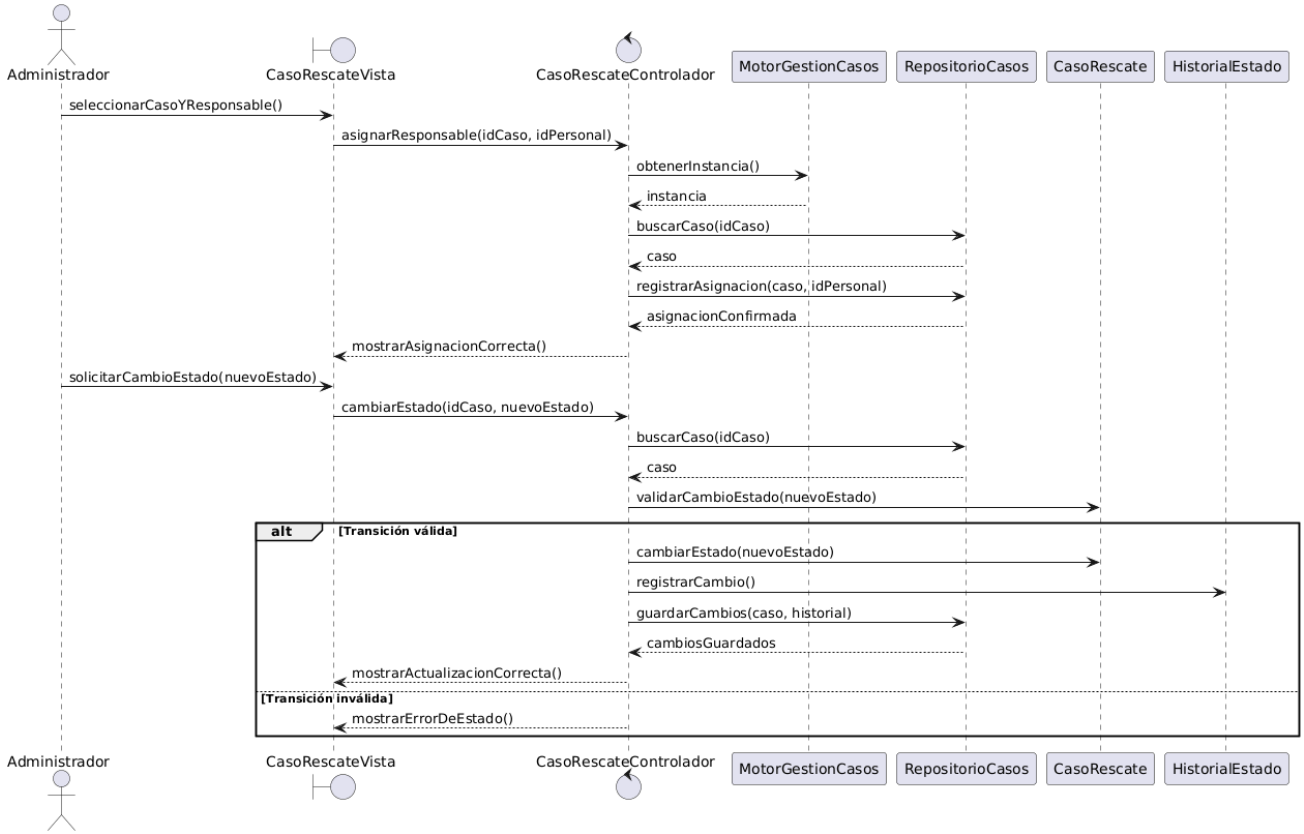
Este escenario combina MVC, Factory Method, Singleton y Observer.



Descripción del proceso:

1. El ciudadano completa el formulario de rescate.
2. La vista envía los datos al controlador.
3. El controlador valida la información recibida.
4. Se solicita la creación del expediente apropiado mediante la fábrica correspondiente.
5. El repositorio guarda el caso y los datos asociados.
6. Solo si el guardado es exitoso, se genera la notificación del nuevo caso.
7. La vista informa al ciudadano si el registro se completó o si ocurrió un error.

Observación técnica: en una implementación completa, el controlador determinaría qué fábrica concreta corresponde a la especie seleccionada. Para ello podría usar un registro de fábricas o un servicio de creación. El método crearExpediente() de la secuencia representa la operación de la fábrica seleccionada, no una clase genérica que cree automáticamente cualquier especie sin conocer su configuración.



9.2. Secuencia: asignar un responsable y actualizar el estado

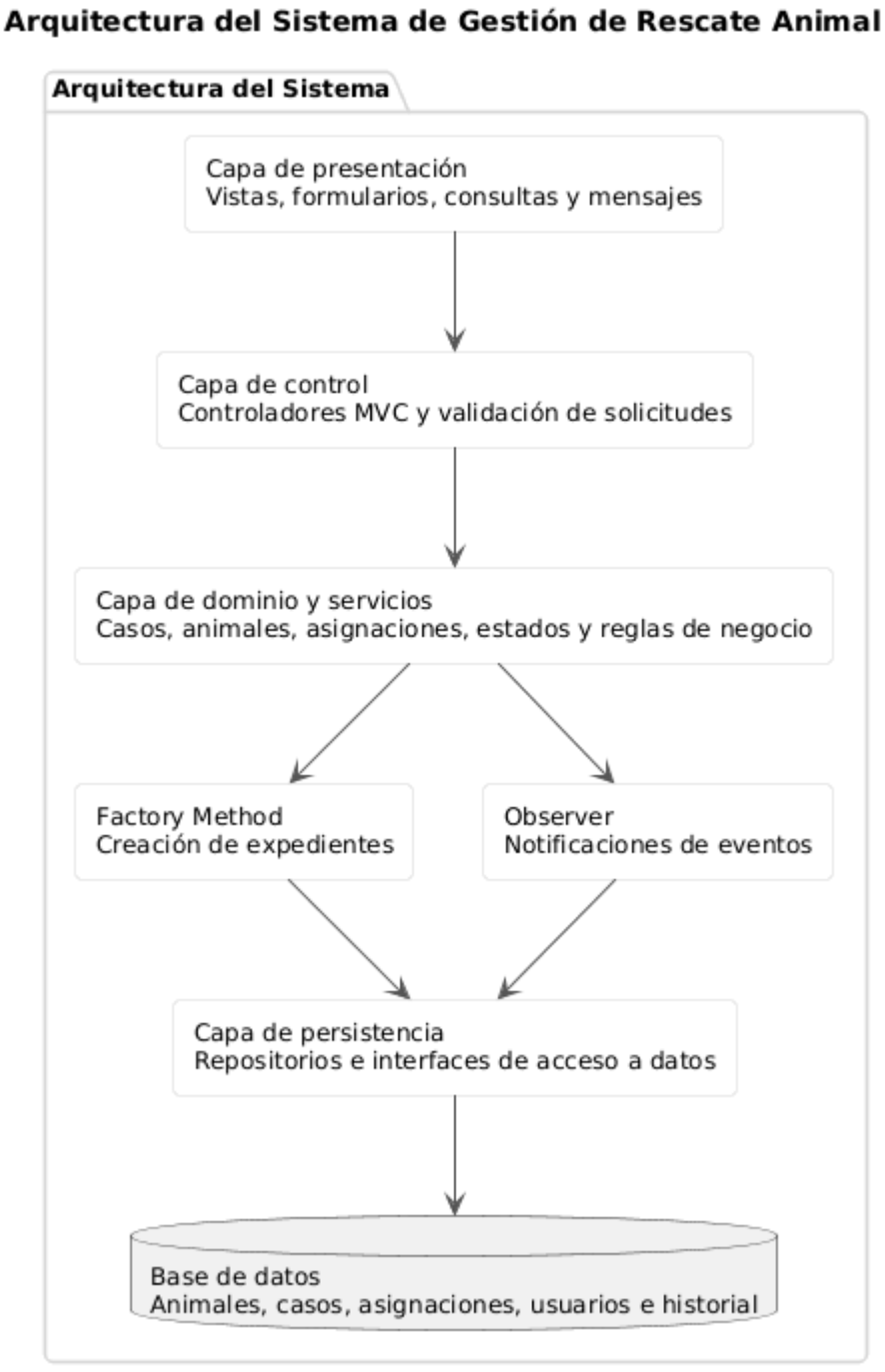
Descripción del proceso:

1. El administrador selecciona un caso y el responsable.
2. El controlador consulta el caso y registra la asignación.
3. Cuando se solicita un cambio de estado, el sistema recupera el caso correspondiente.
4. La transición se valida antes de modificar el estado.
5. Si es válida, se registra el nuevo estado y se conserva el historial.
6. Si es inválida, se informa el error sin registrar un cambio incorrecto.

Para garantizar integridad en la implementación, la modificación del caso y la inserción del historial deberían ejecutarse dentro de una transacción de base de datos. Así se evita que el estado cambie sin que se conserve el registro correspondiente.

10. Arquitectura general del sistema

La arquitectura propuesta separa la aplicación en capas. Cada capa tiene una responsabilidad definida y se comunica con las demás mediante interfaces y operaciones explícitas.



Coordinación transversal

MotorGestionCasos aplica Singleton dentro del alcance definido para la aplicación. Los registros de cambios de código se administran por separado mediante Git y GitHub.

10.1. Responsabilidades de las capas

Capa de presentación. Contiene las pantallas y los formularios con los que interactúan los usuarios. No debería incluir reglas complejas ni consultas directas a la base de datos.

Capa de control. Recibe las solicitudes, valida la entrada y coordina las operaciones necesarias. En MVC, los controladores enlazan la vista con los servicios del dominio.

Capa de dominio y servicios. Contiene las reglas principales: validación de estados, registro de asignaciones, administración de casos y operaciones relacionadas con los animales.

Capa de persistencia. Encapsula el almacenamiento y la recuperación de información. El resto del sistema debería comunicarse mediante contratos de repositorio en lugar de depender de detalles de una base de datos específica.

Base de datos. Conserva la información necesaria para mantener la continuidad del proceso y consultar el historial de cada caso.

10.2. Integración de Gitflow en la arquitectura de desarrollo

Gitflow no es una capa que se ejecute dentro del sistema. Es un flujo de trabajo para organizar el desarrollo del código fuente y revisar cómo se incorporan los cambios.

Por tanto, se representa como un proceso paralelo al diseño de la aplicación:

Las ramas feature/* contienen el desarrollo de funcionalidades específicas.

La rama develop reúne los cambios integrados para la siguiente versión.

La rama master, como la denomina la guía, representa el código estable o preparado para una entrega.

Las ramas de preparación de versión o corrección urgente podrían añadirse si el alcance del proyecto lo requiere, aunque el requisito básico solicita master, develop y ramas feature/*.

11. Organización del trabajo mediante Gitflow

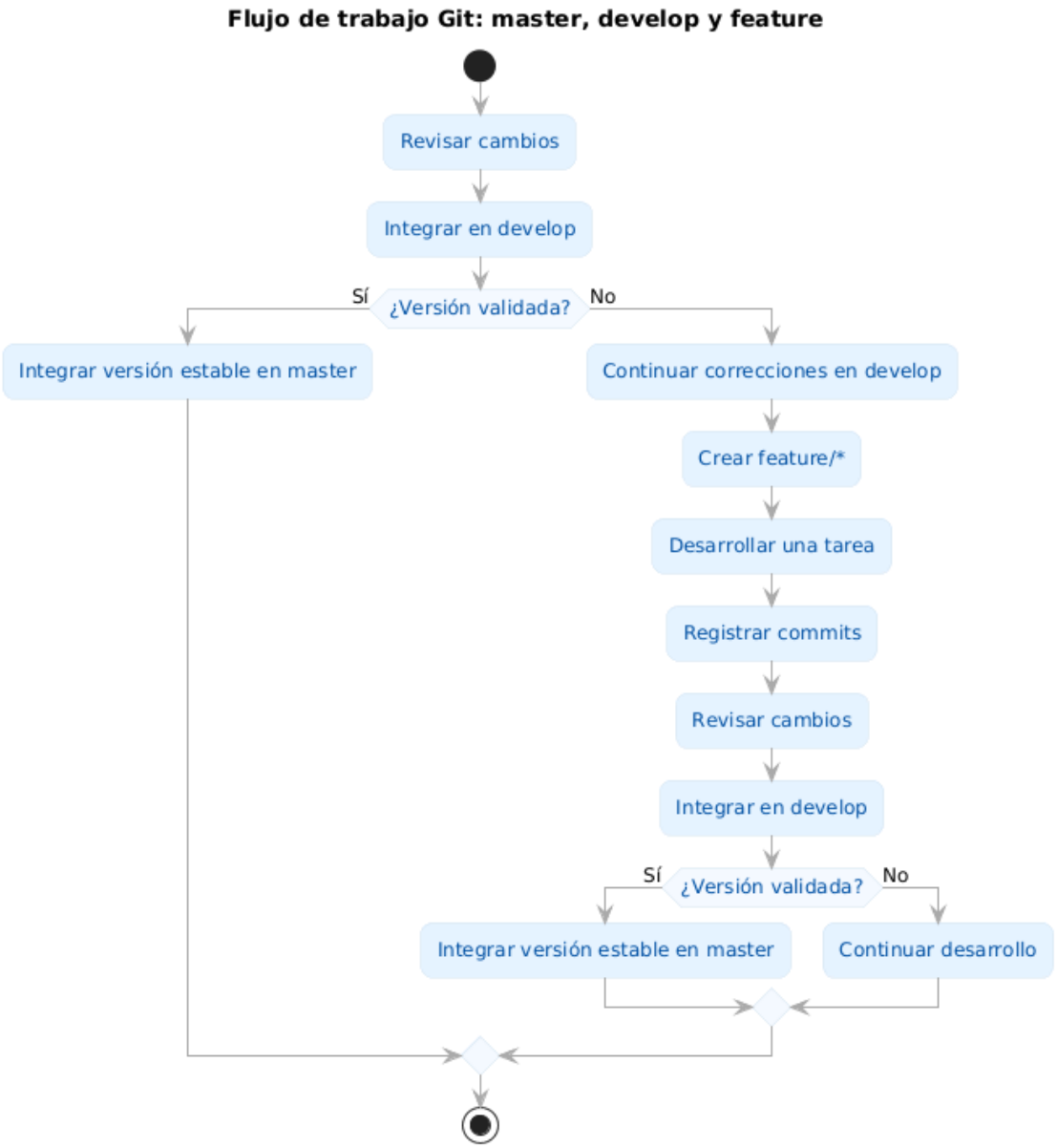
Aunque la creación del repositorio real queda pendiente, es necesario definir cómo se trabajaría con Gitflow para que el diseño incluya la trazabilidad solicitada.

12.1. Ramas principales

Rama	Finalidad	Ejemplo de actividad
master	Mantener la versión estable.	Incorporar una entrega validada.
develop	Integrar funcionalidades en desarrollo.	Reunir los cambios revisados.
feature/analisis-requisitos	Desarrollar el análisis de requisitos.	Documentar actores y requerimientos.
feature/diagramas-uml	Elaborar los diagramas.	Añadir casos de uso y clases.
feature/patrones-diseno	Documentar los patrones.	Incorporar Singleton, Observer y Factory Method.
feature/arquitectura	Definir la arquitectura.	Documentar capas y dependencias.

Las cuatro ramas feature/* son ejemplos de nombres propuestos, no ramas que ya existan en GitHub.

12.2. Flujo de trabajo propuesto



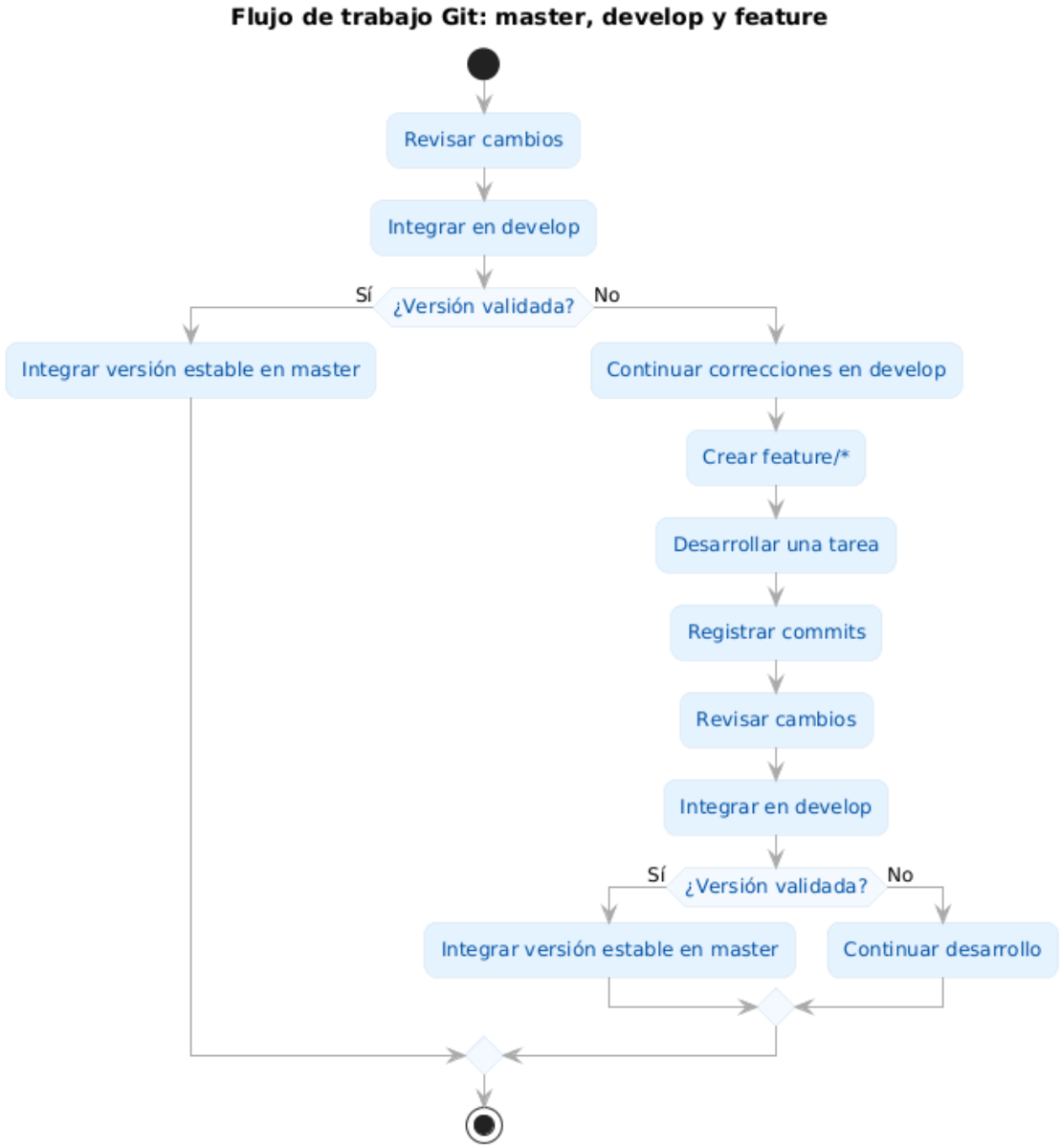
12.3. Secuencia de trabajo recomendada

a. Crear la estructura inicial del proyecto y documentar su objetivo.
b. Crear una rama feature/ para cada grupo de tareas.
c. Realizar cambios pequeños y registrar commits con mensajes descriptivos.
d. Revisar que los diagramas y documentos estén completos antes de integrarlos.
e. Incorporar las ramas terminadas a develop.
f. Verificar la coherencia general del proyecto.

g. Integrar a master únicamente una versión que se haya revisado y validado.

12.4. Ejemplos de mensajes de commit

Los siguientes son ejemplos de mensajes que podrían utilizarse durante el desarrollo:



12.5. Justificación del uso de Gitflow

Gitflow permite separar el trabajo en funcionalidades y mantener una rama estable diferenciada del desarrollo en curso. En este proyecto facilitaría organizar los diagramas, la documentación de los patrones y las futuras funcionalidades del sistema.

La trazabilidad también ayuda a identificar cuándo se incorporó una modificación y qué parte del trabajo cambió. Esto resulta especialmente útil cuando se corrige un diagrama o se ajusta una clase para mantener la coherencia entre los requisitos, la arquitectura y los patrones.

Para que la evidencia sea válida, el historial de ramas y commits debe provenir del repositorio real una vez que se haya creado y utilizado.

13. Decisiones técnicas y justificación

Para que el diseño no sea solamente una colección de diagramas, se establecen las siguientes decisiones:

Decisión	Justificación
Separar Animal de CasoRescate	Un animal puede aparecer en distintos procesos de atención a lo largo del tiempo.
Usar una clase abstracta ExpedienteAnimal	Permite reutilizar operaciones comunes y diferenciar información según la especie.
Mantener el historial de estados	Facilita conocer la evolución del caso y quién realizó cada cambio.
Registrar las asignaciones como entidades	Permite conservar responsables, fechas y funciones de atención.
Aplicar Observer a las notificaciones	Desacopla el registro del caso de los destinatarios de las notificaciones.
Utilizar Factory Method para los expedientes	Centraliza la creación de objetos especializados.
Separar presentación y dominio mediante MVC	Evita mezclar formularios, reglas de negocio y persistencia.

Utilizar repositorios	Facilita sustituir el mecanismo de almacenamiento y realizar pruebas.
Definir un flujo Gitflow	Organiza las modificaciones y conserva la trazabilidad del desarrollo.

13.1. Consideraciones para la implementación futura

a. Los campos obligatorios de cada formulario.
b. Las transiciones válidas entre los estados del caso.
c. Los permisos específicos de ciudadanos, voluntarios, veterinarios y administradores.
d. El procedimiento para validar que un animal puede pasar al proceso de adopción.
e. La estructura de tablas o colecciones para persistir la información.
f. El tratamiento de errores en el almacenamiento y las notificaciones.
g. Las pruebas para verificar las reglas de negocio y los patrones.

El diseño no establece todavía un lenguaje de programación, un framework ni un motor de base de datos obligatorio. Esas decisiones deben adoptarse cuando se conozcan las condiciones técnicas de la implementación.

14. Conclusiones

El análisis del Sistema de Control de Rescate y Adopción Animal permitió identificar las necesidades principales de registro, clasificación, asignación y seguimiento de los animales rescatados. La definición de actores, requisitos y reglas de negocio proporciona una base para organizar el desarrollo posterior.

La elaboración de los diagramas UML mediante la herramienta [PlantText](https://www.planttext.com/) facilitó la representación visual de la estructura del sistema, sus clases, atributos, métodos y relaciones. Estos diagramas contribuyen a comprender la organización de los componentes antes de iniciar su implementación.

La separación entre `Animal`, `CasoRescate` y `ExpedienteAnimal` mejora la claridad del modelo de dominio. Esta estructura permite conservar la identidad del animal, administrar sus procesos de rescate y mantener información específica según su especie.

Los patrones Singleton, Observer y Factory Method responden a necesidades diferentes: coordinación de un componente central, notificación de eventos y creación especializada de expedientes. MVC complementa estas soluciones al separar la presentación, el control y el modelo de negocio.

La incorporación de principios SOLID y una arquitectura por capas favorece la mantenibilidad y la ampliación del sistema. Sin embargo, sus beneficios dependerán de que la implementación respete las responsabilidades y dependencias definidas.

Gitflow proporciona una estrategia para organizar el desarrollo y mantener la trazabilidad de los cambios. La propuesta establece las ramas y el proceso de trabajo, pero su cumplimiento deberá demostrarse mediante un repositorio real y su historial.

Evidencias

Anexo

Capturas del proceso del de subida de la primera versión del trabajo al repositorio de GitHub desde la computadora con el terminal de Git Bash Here

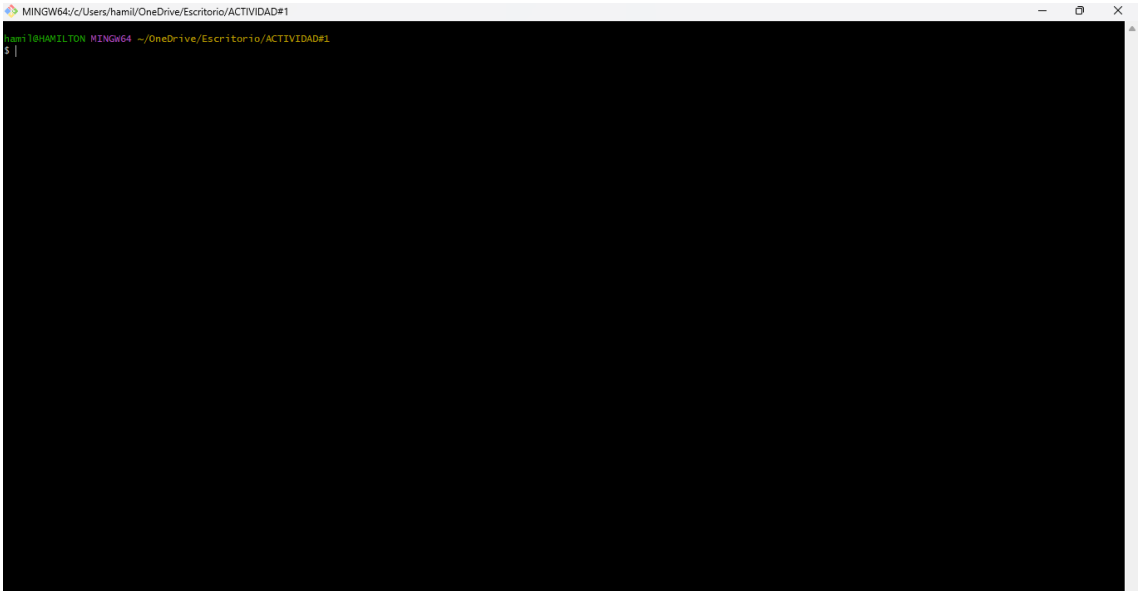


Ilustración 2: Terminal Git Bash Here

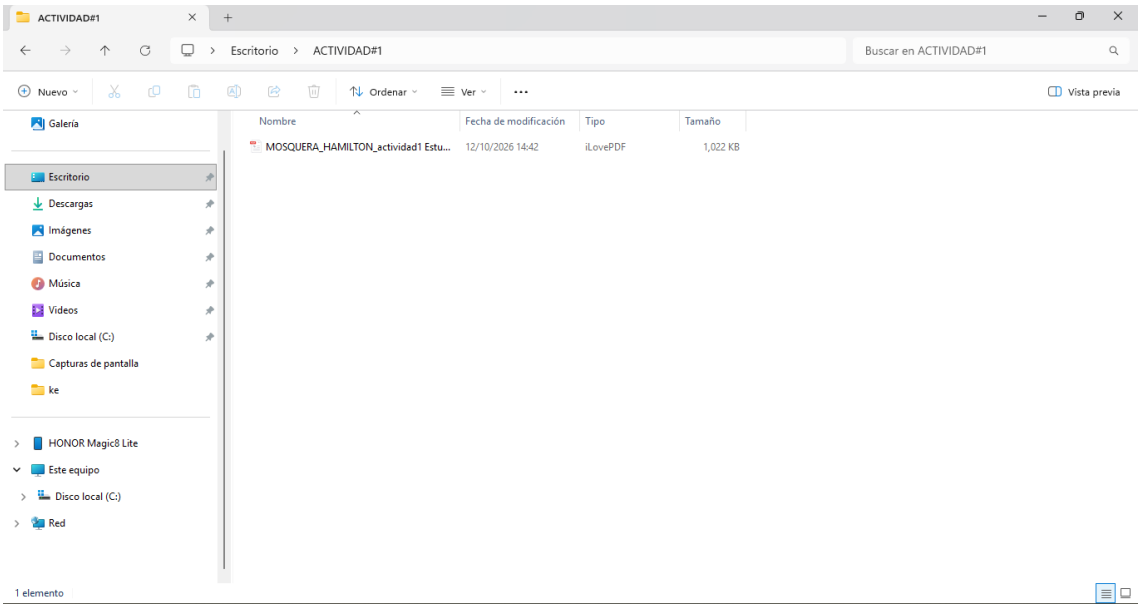
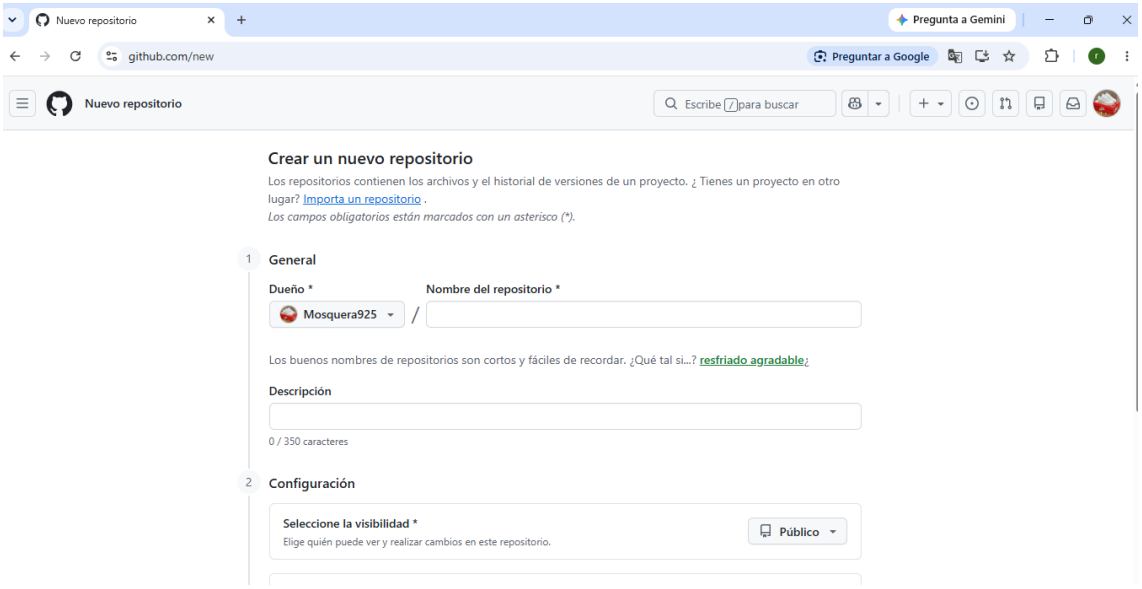


Ilustración 3: Trabajo de la unidad#1 que se subirá al repositorio de GitHub, mediante el terminal de Git Bash Here

Creación del repositorio



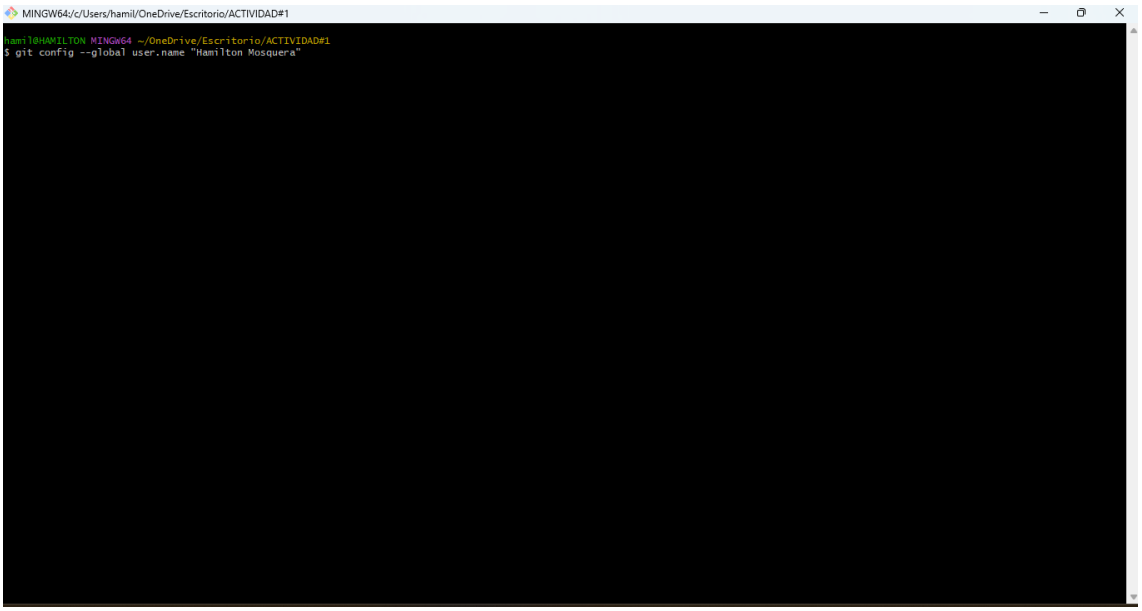


Ilustración 4: Asignando nombre

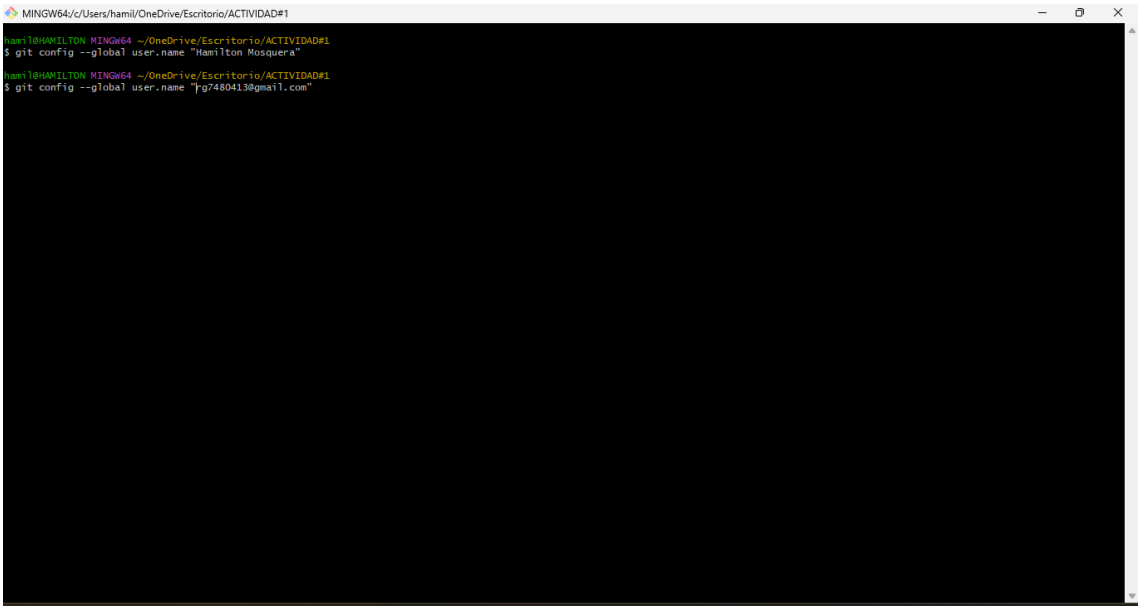


Ilustración 5: : Asignando correo electrónico



Ilustración 6: Comprobando nombre y correo asignado

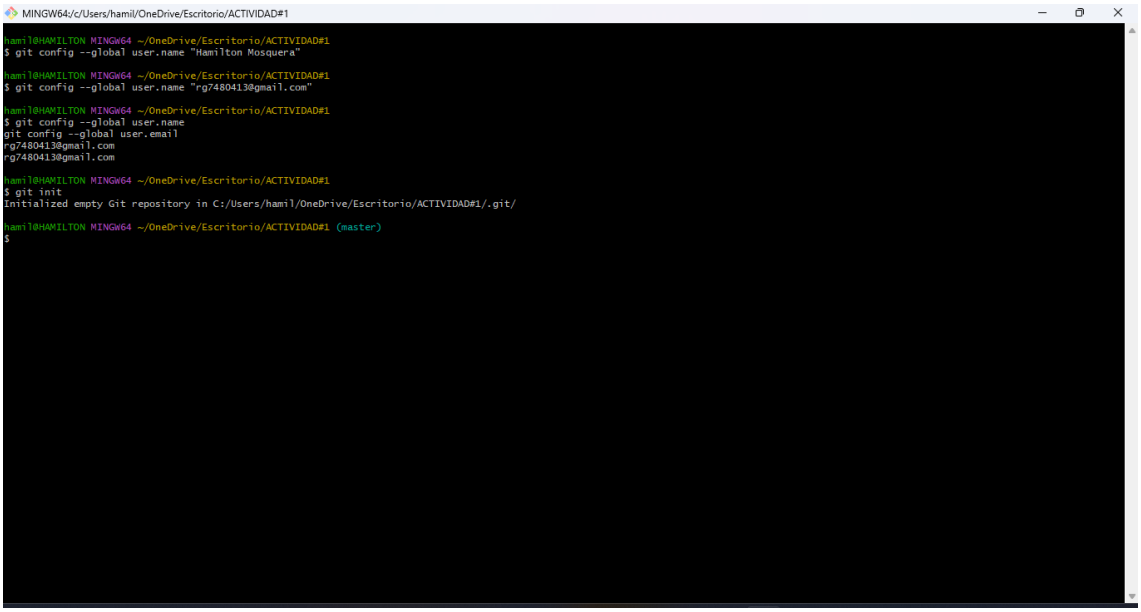


Ilustración 7: Inicialización con el comando git init

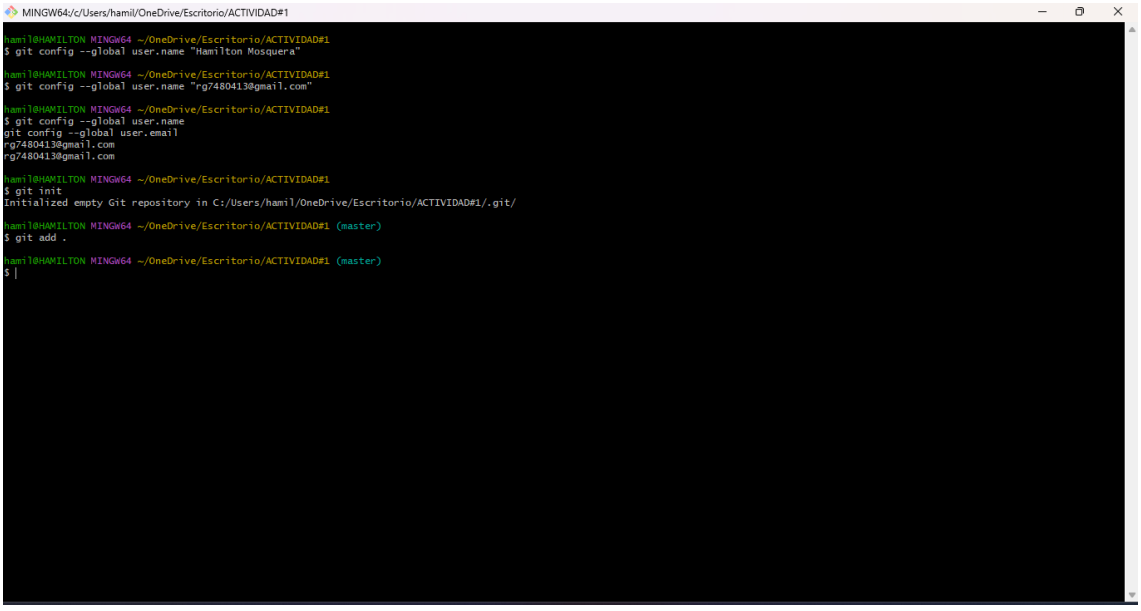


Ilustración 8: Preparación de los archivos con el comando git add .

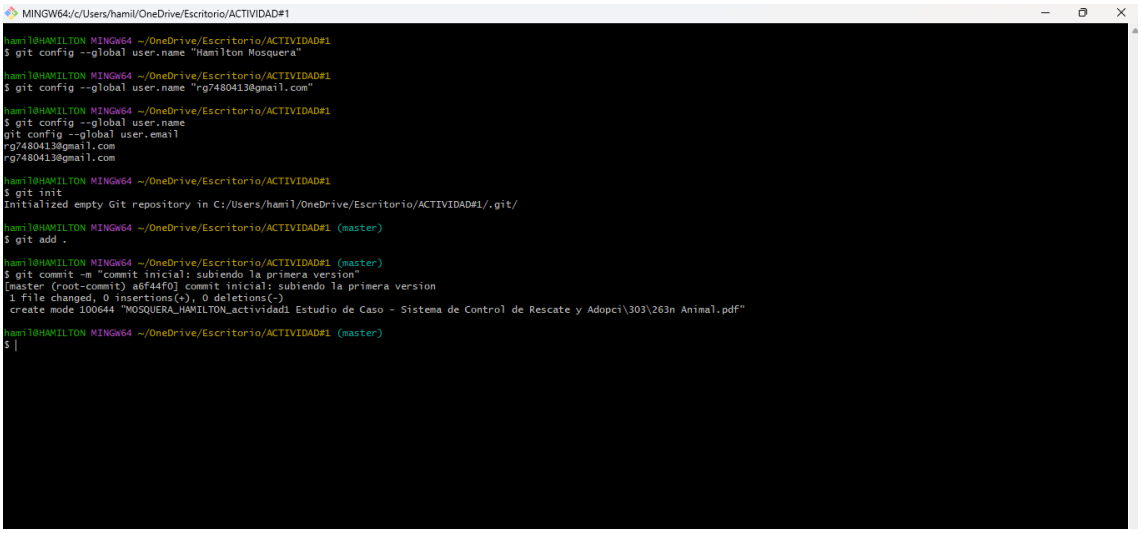


Ilustración 9: Subiendo la primera versión la cual se basa solo en documentación de la primera unidad#1



Ilustración 10: Comandos del repositorio creado para subir la documentación de la primera versión de la actividad#1

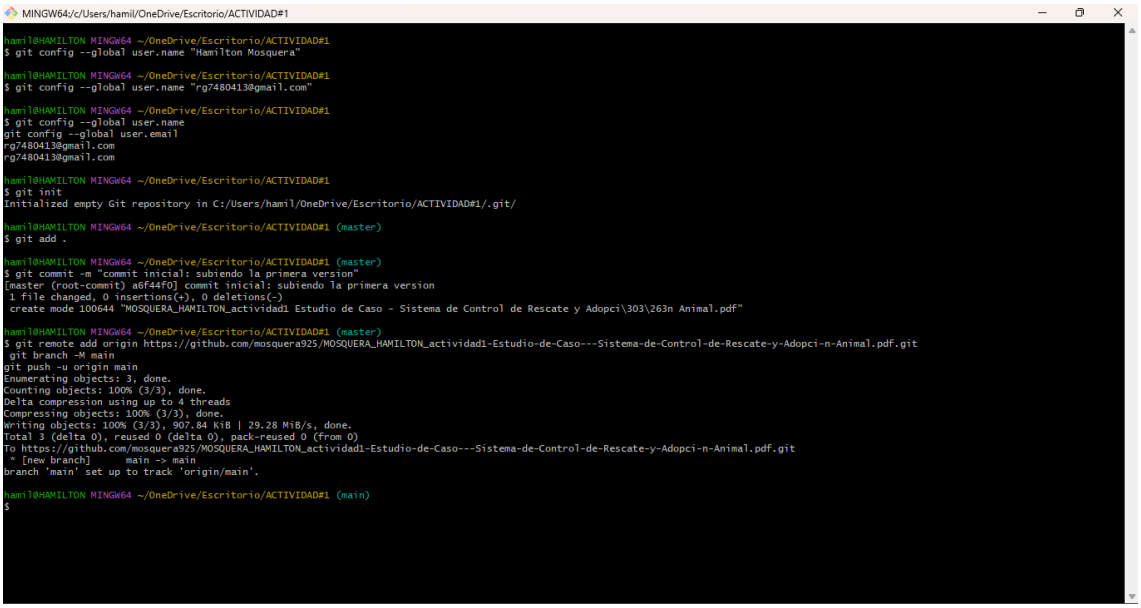
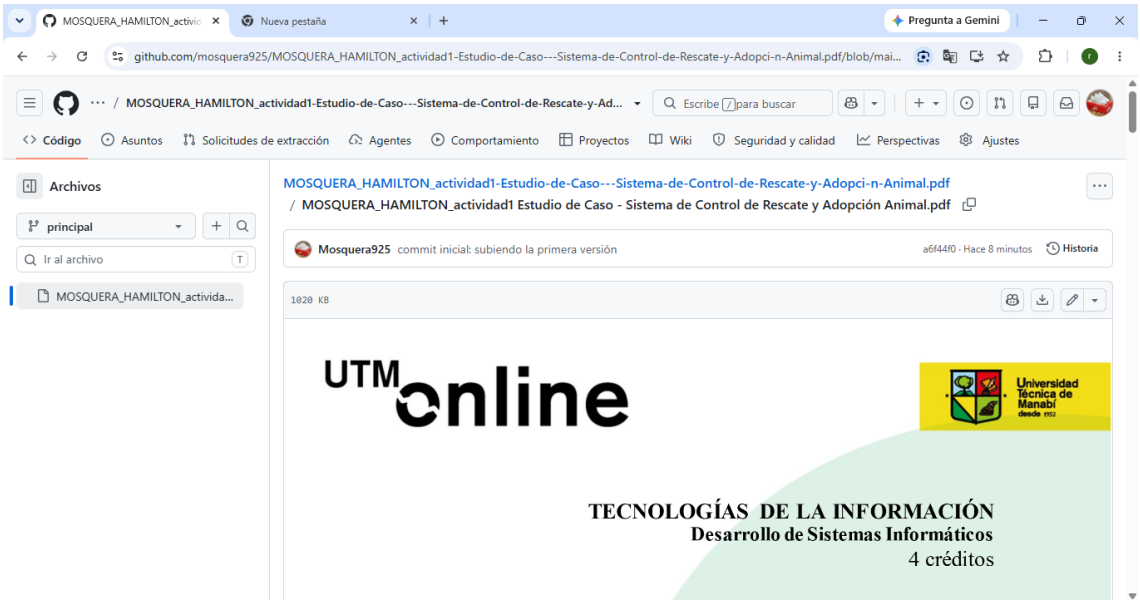


Ilustración 11: Documentación subida al repositorio



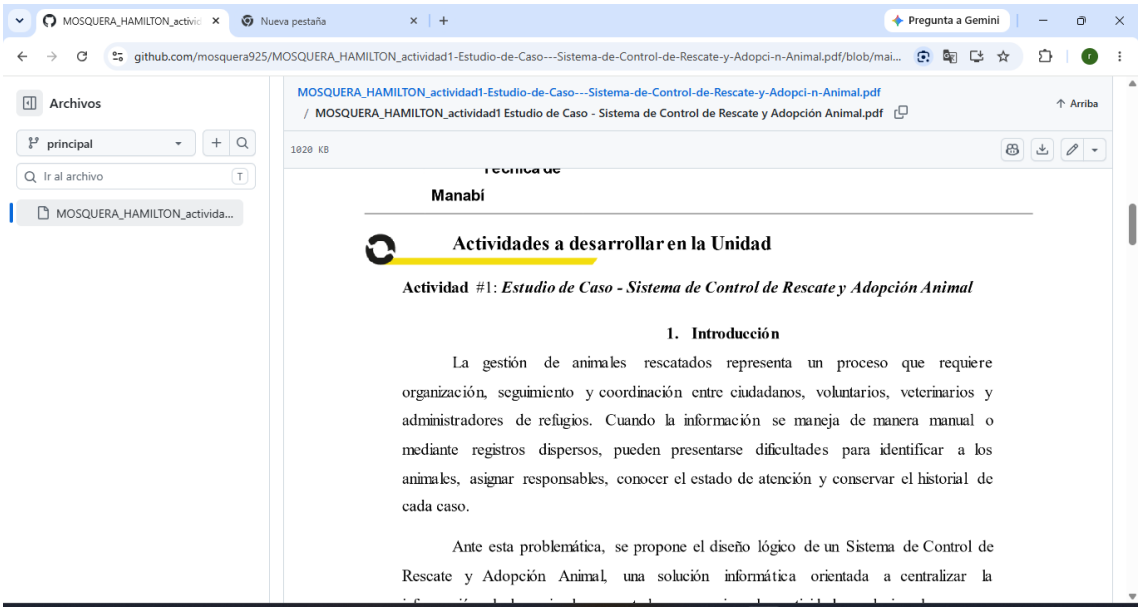


Ilustración 12: Repositorio listo documentación implementada

Enlace al repositorio publico

https://github.com/mosquera925/MOSQUERA_HAMILTON_actividad1-Estudio-de-Caso---Sistema-de-Control-de-Rescate-y-Adopci-n-Animal.pdf/blob/main/MOSQUERA_HAMILTON_actividad1%20Estudio%20de%20Caso%20-%20Sistema%20de%20Control%20de%20Rescate%20y%20Adopci%C3%B3n%20Animal.pdf

15. Bibliografía

PlantText. (12 de octubre de 2026). *Youtube*. Obtenido de Youtube: https://www.youtube.com/watch?v=yI4AA7oXSDg